COL761 Assignment 2

Route Blocking for Reducing Forest Fire Spread

Dharavath Ashvitha , Adwaith S Raj , UDIT
Entry Number: 2022CS11132 , 2022ME11312 , 2025SIY7621

March 2026

1 Introduction

Wildfire propagation can be modeled as a stochastic diffusion process over a directed probabilistic graph. Due to limited resources, only a fixed number of routes can be blocked to reduce fire spread. The goal is to identify a subset of routes whose removal minimizes the expected number of burned regions. This problem is closely related to influence minimization in network diffusion models.

2 Task 1: Optimization Formulation

Let the forest be modeled as a directed probabilistic graph:

$$G = (V, E, p)$$

where V represents forest regions, E represents fire spread routes, and $p(u, v)$ denotes the probability of fire spreading from u to v.

Let $A_0 \subseteq V$ denote the initial burning regions. Fire spreads according to the Independent Cascade model. Let A_∞ denote the set of burned regions at termination.

Define binary decision variables:

$$x_e = \begin{cases} 1 & \text{if edge } e \text{ is blocked,} \\ 0 & \text{otherwise} \end{cases}$$

Let $R = \{e \in E : x_e = 1\}$.

The expected spread is:

$$\sigma(R) = \mathbb{E}[|A_\infty|]$$

The optimization problem is:

$$\min_{R \subseteq E, |R|=k} \sigma(R)$$

subject to:

$$\sum_{e \in E} x_e = k, \quad x_e \in \{0, 1\}.$$

3 Task 2: NP-Hardness

We prove NP-hardness via reduction from the Set Cover problem.

Given universe $U = \{u_1, \ldots, u_n\}$ and subsets $S_1, \ldots, S_m$, construct graph:

- Source node s,
- Node v_i for each subset S_i,
- Node w_j for each element u_j,
- Edge (s, v_i) with probability 1,
- Edge (v_i, w_j) with probability 1 iff $u_j \in S_i$.

Let $A_0 = \{s\}$. Blocking edge (s, v_i) corresponds to selecting subset S_i.

Expected spread equals number of uncovered elements. Thus minimizing spread by blocking k edges corresponds to selecting k subsets covering maximum elements.

Since Set Cover is NP-hard, the route blocking problem is NP-hard.

4 Task 3: Structural Properties

4.1 Monotonicity

Let $R_1 \subseteq R_2$. Blocking additional edges reduces reachable nodes, hence:

$$\sigma(R_1) \geq \sigma(R_2).$$

Thus $\sigma(R)$ is monotone decreasing.

4.2 Submodularity

Define spread reduction:

$$f(R) = \sigma(\emptyset) - \sigma(R).$$

For $R_1 \subseteq R_2$ and $e \notin R_2$:

$$f(R_1 \cup \{e\}) - f(R_1) \geq f(R_2 \cup \{e\}) - f(R_2).$$

Thus $f(R)$ is monotone submodular. Since

$$\sigma(R) = \sigma(\emptyset) - f(R),$$

$\sigma(R)$ is supermodular.

5 Task 4: Algorithm Design

We propose a greedy algorithm accelerated using the **CELF (Cost-Effective Lazy Forward)** optimization [1], which exploits submodularity to avoid redundant marginal gain recomputations.

5.1 Key Idea

Since $f(R) = \sigma(\emptyset) - \sigma(R)$ is monotone submodular, the greedy algorithm that iteratively selects the edge with the highest marginal gain achieves a $(1 - 1/e)$ approximation guarantee. CELF reduces the number of expensive simulations by lazily reusing marginal gain estimates.

5.2 Pseudocode

Algorithm 1 CELF-Greedy Route Blocking

Require: Graph $G = (V, E, p)$, seed set A_0, budget k, simulations r

Ensure: Blocked route set R with $|R| = k$

```
R ← ∅
Initialize priority queue Q
for each e ∈ E do
    Δ_e ← σ̂(∅) − σ̂({e})                          ▷ Estimate via r simulations
    flag_e ← 0                                    ▷ Round last updated
    Push (e, Δ_e, flag_e) into Q (max-heap on Δ_e)
end for
for i = 1 to k do
    updated ← FALSE
    while ¬updated do
        (e*, Δ_e*, flag_e*) ← Q.top(); Q.pop()
        if flag_e* = i then
            R ← R ∪ {e*}                          ▷ Gain is current; select e*
            updated ← TRUE
        else
            Recompute Δ_e* ← σ̂(R) − σ̂(R ∪ {e*})
            flag_e* ← i
            Push (e*, Δ_e*, i) back into Q
        end if
    end while
end for
return R
```

5.3 Computational Complexity

Let $|E| = m$. Estimating $\hat{\sigma}$ via r simulations over a graph with n nodes costs $O(r(n+m))$.

- **Initialization:** Computing marginal gains for all edges requires $O(m)$ simulation calls, giving $O(m \cdot r(n+m))$.
- **Selection loop:** In the worst case, each of the k rounds recomputes gains for $O(m)$ edges before finding a current entry, but submodularity ensures the amortized number of recomputations is $O(m)$ total across all rounds [1].
- **Overall complexity:** $O\big(m \cdot r(n+m)\big)$, compared to $O\big(k \cdot m \cdot r(n+m)\big)$ for naive greedy.

5.4 Approximation Guarantee

Theorem. The greedy algorithm achieves a $\left(1 - \frac{1}{e}\right)$ approximation of the optimal spread reduction.

Proof. Since $f(R) = \sigma(\emptyset) - \sigma(R)$ is a non-negative, monotone, submodular set function with $f(\emptyset) = 0$, the classical result of Nemhauser et al. [2] guarantees that the greedy algorithm, which selects at each step the element with maximum marginal gain, satisfies:

$$f(R_{\text{greedy}}) \;\geq\; \left(1 - \frac{1}{e}\right) f(R^*),$$

where R^* is the optimal solution. Equivalently:

$$\sigma(\emptyset) - \sigma(R_{\text{greedy}}) \;\geq\; \left(1 - \frac{1}{e}\right) \big(\sigma(\emptyset) - \sigma(R^*)\big).$$

CELF does not alter the selection order relative to standard greedy; it only reduces the number of oracle calls by exploiting the submodularity property to skip recomputations. Hence the same approximation guarantee holds. ■

References

[1] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance, *Cost-effective outbreak detection in networks*, KDD, 2007.

[2] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, *An analysis of approximations for maximizing submodular set functions*, Mathematical Programming, 1978.